



Figure 2: Single in-memory process

Integrated workflow and architecture

A common architecture uses Polars as the data preparation layer and DuckDB as the analytical query engine. Raw data is ingested and transformed using Polars' lazy DataFrame API, then exposed to DuckDB for SQL-based analysis. Data exchange typically happens through Arrow-backed memory by registering Polars DataFrames or LazyFrames as DuckDB relations. Execution remains fully in-process, avoiding network overhead or intermediate file writes.

In practice, Polars is best suited for operations that are awkward or verbose in SQL, such as complex column logic, feature engineering, and data cleaning, while DuckDB is ideal for multi-table joins, aggregations, and reporting queries. A common anti-pattern is using DuckDB for row-by-row transformations, better expressed in Polars, or eagerly materialising Polars DataFrames too early. Both negate the benefits of lazy execution and increase memory pressure.

A hands-on example

In this section, we will build a complete analytical pipeline using Polars for fast, vectorised feature engineering and DuckDB for SQL-based analytics. All this runs in-process on a CSV file.

Load the CSV with Polars (Lazy): We start by lazily scanning the CSV. This enables Polars to build an optimised query plan instead of loading the entire dataset immediately.

```
import polars as pl
students = (pl.scan_csv("/content/StudentPerformance.csv")
    .select([
        pl.col("Hours Studied").alias("hours_studied"),
        pl.col("Previous Scores").alias("previous_score"),
        pl.col("Extracurricular Activities").
alias("extracurricular"),
        pl.col("Sleep Hours").alias("sleep_hours"),
        pl.col("Sample Question Papers Practiced").
alias("practice_papers"),
```

```
pl.col("Performance Index").alias("performance"]]))
```

At this stage, no data has been loaded. Polars has only built a logical execution plan.

Feature engineering with Polars: Now we enrich the dataset with derived features:

- Convert Yes/No extracurriculars to 1/0
- Compute performance efficiency
- Compute total preparation effort
- Bucket students by sleep quality

```
features_lazy = (students
    .with_columns([
        pl.when(pl.col("extracurricular") == "Yes")
        .then(1)
        .otherwise(0)
        .alias("has_extracurricular"),
        (pl.col("performance") / pl.col("hours_studied"))
        .alias("performance_per_hour"),
        (pl.col("hours_studied") + pl.col("practice_papers")
        * 2)
        .alias("total_effort"),
        pl.when(pl.col("sleep_hours") >= 8).then(pl.lit("Well
        Rested"))
        .when(pl.col("sleep_hours") >= 6).then(pl.
        lit("Moderate Sleep"))
        .otherwise(pl.lit("Sleep Deprived"))
        .alias("sleep_category"]]))
```

This entire feature pipeline is still lazy. Polars will optimise it before execution.

Execute the Polars query: Now we materialise the dataset:

```
features = features_lazy.collect()
```

Polars executes the optimised query plan and produces a columnar DataFrame backed by Apache Arrow.